Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales

Sistemas de Computación

Trabajo Práctico N° 1 - Rendimiento

Alumno: Gastón Yomaha

Docentes: Javier Jorge
Miguel Solinas

29 de marzo de 2026

Índice

1 Benchmarks 2

2 Time Profiling 2

3 Variación de Frecuencia en Microcontrolador 4

1. Benchmarks

Se utilizan para calcular el rendimiento de alguna tarea, midiendo el tiempo que los diversos computadores tardan en ejecutar los programas que realmente el usuario va a utilizar posteriormente. A continuación se muestra un ejemplo de una tarea cotidiana y el benchmark que la representa.

Tarea	Benchmark
Prender la PC	Tiempo en cargar correctamente un documento luego de encender la PC

Utilizando el sitio https://openbenchmarking.org/test/pts/build-linux-kernel-1.15.0, se obtienen los datos sobre cuánto tiempo tarda cada procesador en ejecutar 2 tareas: "defconfig" y "allmodconfig".
Se utilizarán los siguientes procesadores:
Intel Core i5-13600K (Intel)
AMD Ryzen 9 5900X 12-Core (AMD 1)
AMD Ryzen 9 7950X 16-Core (AMD 2)
Para realizar las comparaciones, se elige como procesador base el Intel.

Tiempo [s]: Tiempo de ejecución de la tarea
Rendimiento: 1 sobre el Tiempo
Speedup: Rendimiento de algún procesador dividido por el Rendimiento del procesador base
Costo: Precio aproximado en dólares de cada procesador (se expresan relativos al procesador base)
Eficiencia de Costo: Speedup sobre Costo
Núcleos: Cantidad de Núcleos de cada procesador (se expresan relativos al procesador base)
Eficiencia de Núcleos: Speedup sobre Núcleos

1) defconfig:

Procesador	Tiempo	Rendimiento	Speedup	Costo	Ef. de Costo	Núcleos	Ef. de Núcleos
Intel	72	0.013889	1	285 (1)	1	14 (1)	1
AMD 1	76	0.013158	0.947	257 (0.902)	1.05	12 (0.857)	1.105
AMD 2	50	0.020000	1.440	480 (1.684)	0.855	16 (1.143)	1.26

2) allmodconfig:

Procesador	Tiempo	Rendimiento	Speedup	Costo	Ef. de Costo	Núcleos	Ef. de Núcleos
Intel	846	0.001182	1	285 (1)	1	14 (1)	1
AMD 1	901	0.001110	0.939	257 (0.902)	1.041	12 (0.857)	1.1
AMD 2	512	0.001953	1.652	480 (1.684)	0.981	16 (1.143)	1.445

En ambas tareas se puede apreciar que el procesador más eficiente en cuanto a costo es el AMD Ryzen 9 5900X 12-Core (AMD 1) mientras que el más eficiente en cuanto a núcleos es el AMD Ryzen 9 7950X 16-Core (AMD 2).

2. Time Profiling

El procesador que se utiliza es el 11th Gen Intel(R) Core(TM) i7-1165G7 @2.80GHz con 8 GB de memoria RAM. Se utiliza la versión de GCC 13.3.0.
Se utiliizan 2 herramientas distintas para realizar el profiling de los códigos "test_gprof.c" y "test_gprof_new.c":

- gprof: funciona por inyección de código
- pref: funciona por sampling

Siguiendo los tutoriales en el documento "Time profiling.pdf"se llega a los siguientes resultados:

1) gprof:

Primero se crea el archivo “test_gprof”, que agrega pequeñas partes de código a “test_gprof.c”.

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_1
File  Edit  View  Terminal  Tabs  Help
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

gaston@mint-virtual:~$ cd SdeC/Practico_1
gaston@mint-virtual:~/SdeC/Practico_1$ gcc -Wall -pg test_gprof.c test_gprof_new.c -o test_gprof
```

Luego se ejecuta el archivo “test_gprof”, creando el archivo “gmon.out”, desde el cual se puede obtener el archivo con los resultados, el cual se llama “analysis.txt”.

```
gaston@mint-virtual:~/SdeC/Practico_1$ ./test_gprof

 Inside main()

 Inside func1

 Inside new_func1()

 Inside func2
gaston@mint-virtual:~/SdeC/Practico_1$ gprof test_gprof gmon.out > analysis.txt
```

Resultados:

```
analysis.txt
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 60.00      0.12     0.12        1   120.00   120.00  func1
 40.00      0.20     0.08        1    80.00    80.00  func2
  0.00      0.20     0.00        1     0.00     0.00  new_func1
```

```
granularity: each sample hit covers 4 byte(s) for 5.00% of 0.20 seconds

index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0    0.00    0.20                 main [1]
                0.12    0.00       1/1           func1 [2]
                0.08    0.00       1/1           func2 [3]
-----------------------------------------------
                0.12    0.00       1/1           main [1]
[2]     60.0    0.12    0.00       1         func1 [2]
                0.00    0.00       1/1           new_func1 [4]
-----------------------------------------------
                0.08    0.00       1/1           main [1]
[3]     40.0    0.08    0.00       1         func2 [3]
-----------------------------------------------
                0.00    0.00       1/1           func1 [2]
[4]      0.0    0.00    0.00       1         new_func1 [4]
-----------------------------------------------
```

Viendo el código de los archivos .c, ya se podría percibir que la función 1 debería ser la que más tiempo tarde, porque su ciclo for tiene el número más grande, seguido por el ciclo for de la función 2. En cambio, tanto el ciclo for del main como el de la new_func1 son demasiado pequeños en comparación con los 2 primeros.

De hecho, teniendo en cuenta los números exactos de los bucles for, el ciclo de func1 es 1.45 veces más grande que el de func2, y viendo los tiempos esa relación casi que se mantiene (func1 tardó 1.5 veces lo que func2).

2) perf:

Se ejecuta el archivo "test_gprof" para obtener el archivo "perf.data".

```
gaston@mint-virtual:~/SdeC/Practico_1$ sudo perf record ./test_gprof

 Inside main()

 Inside func1

 Inside new_func1()

 Inside func2
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0,116 MB perf.data (2750 samples) ]
gaston@mint-virtual:~/SdeC/Practico_1$ sudo perf report
[sudo] password for gaston:
```

Resultados:

```
Terminal - gaston@mint-virtual: ~/SdeC/Practico_1
File  Edit  View  Terminal  Tabs  Help
Samples: 2K of event 'task-clock:ppp', Event count (approx.): 687500000
Overhead  Command     Shared Object      Symbol
  89,60%  test_gprof  test_gprof         [.] func1
   4,36%  test_gprof  test_gprof         [.] func2
   3,38%  test_gprof  test_gprof         [.] main
   1,31%  test_gprof  [kernel.kallsyms]  [k] irqentry_exit_to_user_mode
   0,44%  test_gprof  [kernel.kallsyms]  [k] handle_softirqs
   0,40%  test_gprof  [kernel.kallsyms]  [k] finish_task_switch.isra.0
   0,29%  test_gprof  [kernel.kallsyms]  [k] _raw_spin_unlock_irqrestore
   0,11%  test_gprof  test_gprof         [.] new_func1
   0,04%  test_gprof  [kernel.kallsyms]  [k] __slab_free
   0,04%  test_gprof  [kernel.kallsyms]  [k] _raw_spin_unlock_irq
   0,04%  test_gprof  test_gprof         [.] __do_global_dtors_aux
```

En este caso perf no es preciso.

3. Variación de Frecuencia en Microcontrolador

El tiempo de programa está dado por la siguiente expresión:

$$T_{prog} = \frac{N°instrucciones \cdot CPI}{f_{CPU}}$$

Donde el Número de Instrucciones y los Ciclos por Instrucción dependen del programa, entonces el tiempo de ejecución depende exclusivamente de la frecuencia de la CPU. Para determinado programa, si se aumenta la frecuencia de la CPU, disminuirá el tiempo de ejecución.